Experiment no.5

Aim: Write a program to demonstrate the concept of non-preemptive scheduling algorithms

Objective : To gain practical experience with designing and implementing concept of operating system such as process management

Software used: Turbo C.

Theory:

The act of determining which process is in the ready state, and should be moved to the running state is known as Process Scheduling.

The prime aim of the process scheduling system is to keep the CPU busy all the time and to deliver minimum response time for all programs

Scheduling fell into one of the two general categories:

Non Pre-emptive Scheduling: When the currently executing process gives up the CPU voluntarily.

Pre-emptive Scheduling: When the operating system decides to favour another process, pre-empting the currently executing process.

There are various CPU Scheduling algorithms such as-

First Come First Served (FCFS)

Shortest Job First (SJF)

Priority Scheduling

Round Robin (RR)

1)FCFS

First in, first out (FIFO), also known as first come, first served (FCFS), is the simplest scheduling algorithm. FIFO simply queues processes in the order that they arrive in the ready queue.
In this, the process that comes first will be executed first and next process starts only after the previous gets fully executed
Easy to understand and implement.
Its implementation is based on FIFO queue.
Poor in performance as average wait time is high.

2)Shortest job first (SJF) or shortest job next, is a scheduling policy that selects the waiting process with the smallest execution time to execute next. SJN is a non-preemptive algorithm.

Shortest Job first has the advantage of having a minimum average waiting time among all scheduling algorithms.

It may cause starvation if shorter processes keep coming. This problem can be solved using the concept of ageing.

It is practically infeasible as Operating System may not know burst time and therefore may not sort them. While it is not possible to predict execution time, several methods can be used to estimate the execution time for a job, such as a weighted average of previous execution times. SJF can be used in specialized environments where accurate estimates of running time are available.

3)Priority Scheduling

Priority scheduling is one of the most common scheduling algorithms in batch systems. Each process is assigned a priority. Process with the highest priority is to be executed first and so on.
Processes with the same priority are executed on first come first served basis.
Priority can be decided based on memory requirements, time requirements or any other resource requirement.

4)Round Robin is a CPU scheduling algorithm where each process is assigned a fixed time slot in a cyclic way.

It is simple, easy to implement, and starvation-free as all processes get fair share of CPU.
One of the most commonly used technique in CPU scheduling as a core.
It is preemptive as processes are assigned CPU only for a fixed slice of time at most.
The disadvantage of it is more overhead of context switching.

Code:

a)FCFS

```
#include<stdio.h>
#include<conio.h>
int main()
{
int n,bt[20],wt[20],tat[20],avwt=0,avtat=0,i,j;
clrscr();
printf("Enter total number of processes(maximum 20):");
scanf("%d",&n);
printf("\nEnter Process Burst Time\n");
for(i=0;i<n;i++)
{
printf("P[%d]:",i+1);
scanf("%d",&bt[i]);
}
wt[0]=0;

//calculating waiting time
for(i=1;i<n;i++)
{
wt[i]=0;
for(j=0;j<i;j++)
wt[i]+=bt[j];
}
printf("Process\t\tBurst Time\twaiting Time\tTurnaround Time");

//calculating Turnaround time
for(i=0;i<n;i++)
{
tat[i]=bt[i]+wt[i];
avwt+=wt[i];
```

```
avtat+=tat[i];
printf("\nP[%d]\t\t%d\t\t%d\t\t%d",i+1,bt[i],wt[i],tat[i]);
 }
avwt/=i;
avtat/=i;
printf("\n\n Average waiting time:%d",avwt);
printf("\n\n Average turn around time:%d",avtat);
getch();

return 0;
}
/*
```

Output:

```
Enter total number of processes(maximum 20):3

Enter Process Burst Time
P[1]:10
P[2]:6
P[3]:5
Process        Burst Time      waiting Time    Turnaround Time
P[1]                10              0               10
P[2]                6               10              16
P[3]                5               16              21

 Average waiting time:8

 Average turn around time:15
*/
```

B)SJF

```
#include<stdio.h>
#include<conio.h>
int main()
{
int n,bt[20],wt[20],tat[20],total=0,i,j,pos,temp,p[20];
float avg_wt,avg_tat;
clrscr();
printf("Enter total number of processes(maximum 20):");
scanf("%d",&n);
printf("\nEnter Process Burst Time\n");
for(i=0;i<n;i++)
{
printf("P%d:",i+1);
scanf("%d",&bt[i]);
p[i]=i+1;    //process number
}

//sorting burst time in ascending order using selection sort
for(i=0;i<n;i++)
{
pos=i;
for(j=i+1;j<n;j++)
{
if(bt[j]<bt[pos])
pos=j;
}
temp=bt[i];
bt[i]=bt[pos];
bt[pos]=temp;
temp=p[i];
p[i]=p[pos];
p[pos]=temp;

//printf("Process\t\tBurst Time\twaiting Time\tTurnaround Time");
wt[0]=0;
```

```
//calculating waiting time
for(i=1;i<n;i++)
{
wt[i]=0;
for(j=0;j<i;j++)
wt[i]+=bt[j];
total+=wt[i];
}
avg_wt=(float)total/n;
total=0;
printf("Process\t\tBurst Time\twaiting Time\tTurnaround Time");

for(i=0;i<n;i++)
{
tat[i]=bt[i]+wt[i];
total+=tat[i];
printf("\np%d\t\t%d\t\t%d\t\t%d",p[i],bt[i],wt[i],tat[i]);
 }
avg_tat=(float)total/n;

printf("\n\n Average waiting time:%f",avg_wt);
printf("\n\n Average turn around time:%f",avg_tat);
getch();
}
return 0;
}
/*
```

Output:

```
Enter total number of processes(maximum 20):5

Enter Process Burst Time
P1:10
P2:5
P3:6
P4:4
P5:12
Process        Burst Time      waiting Time    Turnaround Time
```

p4	4	0	4
p2	5	4	9
p3	6	9	15
p1	10	15	25
p5	12	25	37

Average waiting time:10.600000

Average turn around time:18.000000

*/

Outcome: At the end of this practical, student will be able to design and develpop process management in operating system.

Conclusion: In this way ,we have implemented non-preemptive scheduling algorithms FCFS and SJF.